

Agent 成本与性能优化：Token 预算、限流与成本归因

语言策略：code (Java / Python / TypeScript)

TL;DR

1. Agent 成本首先来自上下文膨胀，其次才是模型单价；先做上下文预算，再谈模型路由。
2. 性能优化顺序：先测 P95 与 TTFT，再优化最慢的工具和最长上下文，最后才换更贵的模型。
3. 任何优化都要绑定成本、延迟、成功率三个指标，防止“省了钱、坏了效果”。

1. 成本公式

```
1 单任务成本 = 输入 Token × 输入单价
2              + 输出 Token × 输出单价
3              + 工具调用成本
4              + 缓存命中节省
5              + 分摊基础设施成本
```

成本归因必须带 task_id、model、prompt_version、toolset_version，否则无法定位是哪个版本变贵。

2. 成本优化杠杆

杠杆	做法	典型收益
上下文预算	每轮计算预算，裁剪低优先级内容	高
模型分级	简单任务走小模型，复杂任务走强模型	高
Prompt Cache	System Prompt 与工具 Schema 固定前缀	中高
工具返回压缩	只保留结构化摘要	高
轮次上限	降低无效循环	中
语义缓存	相似问题直接复用结果	中，需防错
批处理	离线任务合并调用	中

3. 性能优化顺序

1. 建立基线：P50 / P95 / TTFT / TPOT / 单任务 Token；
2. 找出最慢工具并设置超时与并行；
3. 缩短上下文：裁剪历史、压缩工具返回；
4. 流式输出降低首字延迟；
5. 模型路由：强模型只处理复杂任务；
6. 缓存与预取：Prompt Cache、检索预取。

4. 成本与性能门禁

- 单任务成本超预算 130% 告警；
- P95 超过 SLO 120% 告警；
- 成本优化上线前必须过评测门禁，成功率下降即回滚；
- 每月输出成本归因报告：按租户、任务类型、模型、Prompt 版本、工具。

5. Java 版成本追踪与限流

```
1 public record TokenUsage(long inputTokens, long outputTokens, double
2   ↪ inputPrice,
3   double outputPrice, double toolCost) {
4   public double cost() {
5     return inputTokens * inputPrice + outputTokens * outputPrice
6     ↪ + toolCost;
```

```
5   }
6 }
7
8 public class SlidingWindowLimiter {
9   private final int maxRequests;
10  private final long windowMillis;
11  private final Deque<Long> timestamps = new ArrayDeque<>();
12
13  ↪ public SlidingWindowLimiter(int maxRequests, long windowMillis)
14  {
15    this.maxRequests = maxRequests;
16    this.windowMillis = windowMillis;
17  }
18
19  public synchronized boolean tryAcquire(long now) {
20    while (!timestamps.isEmpty() && now - timestamps.peekFirst()
21    ↪ >= windowMillis) {
22      timestamps.pollFirst();
23    }
24    if (timestamps.size() >= maxRequests) return false;
25    timestamps.addLast(now);
26    return true;
27  }
```

6. 上线检查单

- 有单任务 Token 与成本预算；
- 有滑动窗口限流与并发上限；
- 有模型分级路由与降级策略；
- 有成本/延迟/成功率三个维度的回归对比；
- 缓存命中与错误率同时监控。

7. 延伸阅读

- [大模型网关](#)
- [Agent 可观测与追踪](#)
- [Agent 评测体系](#)